

GENERATIVE AI ASSIGNMENT

How Large Language Models (LLMs) Work

Tokenization, Transformers, Attention, Training, Fine-Tuning and
Text Generation

Student: Gull

Topic: How LLMs Work

Level: Level 0–1 Generative AI

Assignment: Written Summary

Date: October 2026

Understanding the basic ideas behind modern Large Language Models

Contents

1	Introduction	2
2	Tokenization	2
2.1	Why Tokenization Matters	3
3	Token IDs and Embeddings	3
4	Transformer Architecture	3
5	Attention	4
5.1	Simple Example	4
6	How an LLM Learns	5
7	Pre-Training	5
8	Fine-Tuning	6
9	Base Model vs. Instruction-Tuned Model	6
10	Training vs. Inference	7
10.1	Training	7
10.2	Inference	7
11	How an LLM Generates Text	8
12	Temperature	8
13	Top-p	9
14	Complete Example: From Prompt to Answer	10
15	Complete Overview	10
16	Written Summary	11
17	Acceptance Criteria Checklist	12
18	Conclusion	12

1 Introduction

Large Language Models (LLMs) are artificial intelligence systems designed to understand and generate human-like text. They are used in chatbots, coding assistants, content-generation tools, translation systems, and many other applications.

An LLM does not process language in exactly the same way as a human. Instead, it processes text as smaller units called **tokens**. The model then uses a neural network to learn patterns and relationships between these tokens.

Modern LLMs are mainly based on the **Transformer** architecture. Transformers use a mechanism called **attention** to understand relationships between different parts of the input.

A simple view of an LLM is:

Text → **Tokens** → **Numbers** → **Transformer** → **Next-token prediction** → **Generated text**

The following sections explain these concepts step by step.

2 Tokenization

Tokenization is the process of breaking text into smaller pieces called **tokens**.

A token can be:

- a complete word,
- part of a word,
- punctuation,
- or another small piece of text.

For example, the sentence:

I love programming.

could be divided conceptually into:

I love programming .

The exact tokens depend on the tokenizer and the model.

Therefore, a token should not be considered exactly the same thing as a word.

For example, a long or uncommon word may be divided into several smaller pieces.

2.1 Why Tokenization Matters

LLMs do not directly perform their calculations on normal human sentences. They process numerical representations of tokens.

The basic process is:

Human Text → **Tokenizer** → **Tokens** → **Token IDs**

Tokenization is important because token counts affect the amount of text a model can process and are also commonly used when measuring API usage.

Important idea:

A token is not necessarily a word. One word can become multiple tokens, and punctuation can also be represented as tokens.

3 Token IDs and Embeddings

After text is divided into tokens, each token is represented using a numeric ID.

For example, conceptually:

"Hello" → Token ID

The actual number depends on the tokenizer being used.

The model then works with numerical representations called **embeddings**.

An embedding can be thought of as a list of numbers that represents a token in a form that the neural network can process.

Conceptually:

"cat" → **Token ID** → **Numerical representation**

The important idea is that the neural network performs its calculations using numbers rather than directly processing English words.

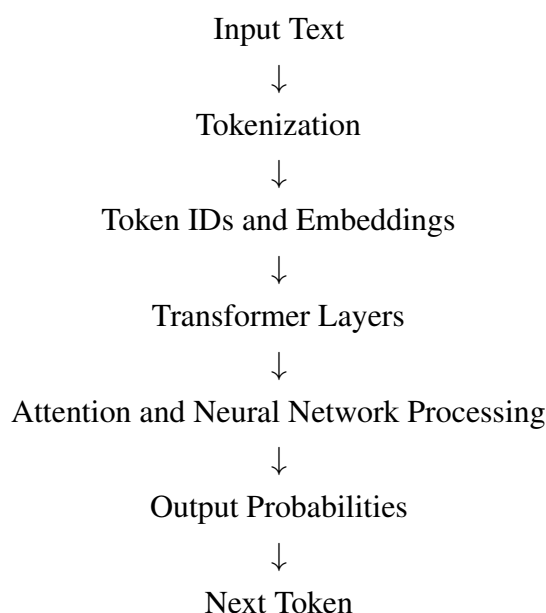
4 Transformer Architecture

The **Transformer** is a neural-network architecture that became the foundation for many modern language models.

The Transformer architecture was introduced in the 2017 research paper *Attention Is All You Need*.

The main idea is to allow the model to process relationships between different tokens using attention.

At a high level:



A Transformer contains multiple processing layers. These layers repeatedly process the token representations and build a richer understanding of the input.

5 Attention

Attention is one of the most important ideas in the Transformer architecture.

Attention allows the model to consider how different tokens relate to one another.

Consider the sentence:

The dog chased the ball because it was excited.

When processing the word “it”, the model needs to consider other words in the sentence to understand the meaning of “it”.

Attention helps the model determine which parts of the surrounding context are relevant.

5.1 Simple Example

Consider:

The student opened the laptop because it was needed for the assignment.

The meaning of “it” depends on the surrounding context.

Attention allows the model to consider relationships between tokens instead of treating every token as completely independent.

In simple words:

Attention helps an LLM determine which other tokens are important when processing a particular token.

6 How an LLM Learns

An LLM learns by processing a very large amount of training data and adjusting its internal numerical parameters.

A simplified training process is:

Training Data → Prediction → Calculate Error → Update Parameters → Repeat

The model contains a very large number of parameters. During training, these parameters are adjusted so that the model becomes better at its learning task.

For example, suppose the training text contains:

The capital of France is Paris.

The model can learn patterns connecting “capital of France” with “Paris”.

The model does not learn from one example only. It processes a very large amount of data and repeatedly updates its parameters.

7 Pre-Training

Pre-training is the large initial training stage of an LLM.

During pre-training, the model learns general language patterns from a very large collection of training data.

A simplified representation is:

Large Dataset → Pre-Training → General Language Model

The model can learn patterns involving:

- vocabulary,
- grammar,
- sentence structure,
- relationships between words,
- common patterns in text,

- programming patterns.

Pre-training provides the model with a broad foundation.

8 Fine-Tuning

Fine-tuning means continuing the training of an already pre-trained model using a smaller dataset focused on a particular task or domain.

For example:

Pre-trained Model → Coding Dataset → Fine-Tuned Model

A coding-focused dataset could contain many examples of programming questions and answers.

Another example is customer support:

Customer:

How can I reset my password?

Support Agent:

You can reset your password by selecting “Forgot Password” on the login page.

Fine-tuning can help make a model more suitable for a particular task, style, or domain.

Pre-Training	Large-scale general training used to build broad language abilities.
Fine-Tuning	Additional training using a smaller dataset focused on a particular task or domain.

9 Base Model vs. Instruction-Tuned Model

A **base model** is a model that has mainly learned general language patterns during its initial training.

Its behavior can be understood as predicting what text is likely to come next.

For example:

The largest planet in our solar system is

The model may predict:

Jupiter

An **instruction-tuned model** receives additional training designed to make it better at following human instructions.

For example:

User:

Explain photosynthesis in simple English.

Instruction-Tuned Model:

Photosynthesis is the process plants use to make food using light, water, and carbon dioxide.

Therefore:

Base Model → Instruction Training → Instruction-Tuned Model

The key difference is that instruction tuning is intended to improve how the model responds to human instructions.

10 Training vs. Inference

Training and inference are two different stages.

10.1 Training

During training, the model learns by updating its parameters.

Data → Prediction → Error → Parameter Update

The parameters change during this process.

10.2 Inference

Inference means using an already-trained model to generate an output.

For example:

Write a Python function that reverses a string.

The trained model processes the input and generates a response.

During normal inference, the model is being used rather than performing the original large-scale parameter-learning process.

Training	The model learns and updates its parameters.
Inference	The trained model is used to generate an output.

11 How an LLM Generates Text

A useful way to understand text generation is to think about **next-token prediction**.

Suppose the user writes:

The sun rises in the

The model produces probabilities for possible next tokens.

A simplified example could be:

Possible Token	Example Probability
east	0.70
morning	0.10
sky	0.05
west	0.03
other tokens	0.12

Suppose the model selects:

east

The new context becomes:

The sun rises in the east

The model then predicts the next token.

This process continues repeatedly until the model reaches an appropriate stopping point or generation limit.

Therefore:

Prompt → Tokens → Transformer → Next-token probabilities → Select token → Repeat →
Response

12 Temperature

Temperature is a setting that affects how random or focused the model's token selection is during generation.

A lower temperature generally makes output more focused and predictable, while a higher temperature allows more variation.

For example:

Token	Probability
blue	0.70
clear	0.15
bright	0.10
dark	0.05

With a lower temperature, the model tends to favor the strongest choice more heavily.

With a higher temperature, less likely choices can become more competitive.

Easy way to remember:

Low temperature → more focused and predictable output.

High temperature → more varied output.

13 Top-p

Top-p, also called **nucleus sampling**, controls which likely tokens are considered during generation.

Instead of considering every possible token, the model keeps the smallest group of high-probability tokens whose combined probability reaches the selected value of p .

For example:

Token	Probability
A	0.50
B	0.25
C	0.15
D	0.05
E	0.05

If:

$$top-p = 0.90$$

then:

$$0.50 + 0.25 + 0.15 = 0.90$$

Therefore, A, B, and C form the smallest group whose combined probability reaches 0.90.

Easy way to remember:

Top-p controls how much of the most likely probability group is considered when choosing the next token.

Temperature and top-p affect generation differently.

Temperature changes the probability distribution, while top-p limits the candidate tokens using cumulative probability.

14 Complete Example: From Prompt to Answer

Consider the following user prompt:

Explain Docker in simple English.

A simplified LLM process would look like this:

1. **Tokenization:** The sentence is divided into tokens.
2. **Token IDs:** The tokens are represented using numerical IDs.
3. **Embeddings:** The IDs are converted into numerical representations that the neural network can process.
4. **Transformer Processing:** The representations pass through Transformer layers.
5. **Attention:** The model considers relationships between relevant tokens.
6. **Prediction:** The model produces probabilities for possible next tokens.
7. **Sampling:** Temperature and top-p can influence which token is selected.
8. **Next Token:** The selected token is added to the generated response.
9. **Repeat:** The model predicts additional tokens one by one.
10. **Final Answer:** The generated tokens form the final readable response.

15 Complete Overview

The complete process can be summarized as follows:

TRAINING STAGE

Large Dataset → Pre-Training → General Model → Fine-Tuning →
Instruction-Tuned Model

INFERENCE STAGE

User Prompt → Tokenization → Token IDs → Embeddings → Transformer →
Attention → Next-Token Probabilities → Temperature / Top-p → Next Token →
Repeat → Final Response

This distinction is important because training and inference are not the same process. Training builds or adapts the model, while inference uses the trained model to produce an output.

16 Written Summary

Large Language Models (LLMs) are AI systems that learn patterns from large amounts of text and use those patterns to generate text. Before an LLM can process a sentence, the text is divided into smaller units called **tokens**. Tokens can be complete words, parts of words, or punctuation. These tokens are converted into numerical IDs and representations that the neural network can process. Tokenization is important because LLMs process tokens rather than raw text, and token counts affect model limits and usage.

Modern LLMs are mainly based on the **Transformer** architecture. Transformers contain layers that use **attention** to understand relationships between tokens. For example, in a sentence containing a pronoun such as “it”, attention helps the model consider relevant words in the surrounding context. This allows the model to build a better representation of the input.

LLMs first go through **pre-training**, where they learn general language patterns from a very large dataset. After that, a model can be **fine-tuned** using a smaller dataset focused on a particular task or domain. A base model mainly learns general language prediction, while an instruction-tuned model receives additional training to follow human instructions more effectively.

During **training**, the model learns by adjusting its internal parameters. During **inference**, the trained model is used to generate an answer. The model generally generates text one token at a time by predicting probabilities for possible next tokens.

Two important generation settings are **temperature** and **top-p**. Temperature controls how focused or varied the generation is. Lower values generally produce more predictable output, while higher values allow more variation. Top-p, also called nucleus sampling, limits the choices to a group of likely tokens whose combined probability reaches the selected value. Together, these concepts provide a basic understanding of how modern LLMs process language, learn from data, and generate responses.

17 Acceptance Criteria Checklist

Requirement	Covered In
Tokenization is explained	Section 2
Why tokenization matters	Section 2
Transformer architecture	Section 4
Attention	Section 5
Pre-training	Section 7
Fine-tuning	Section 8
Training vs. inference	Section 10
Base vs. instruction-tuned models	Section 9
Temperature	Section 12
Top-p	Section 13
Practical examples	Throughout the assignment
Written summary	Section 16

18 Conclusion

Understanding an LLM becomes easier when the complete process is viewed as a sequence of simple steps. Text is first converted into tokens, tokens are represented numerically, and Transformer layers process the information using attention.

During training, the model's parameters are adjusted so that it becomes better at its learning task. Pre-training provides broad language abilities, while fine-tuning can adapt a model for a specific purpose or behavior.

During inference, the trained model generates text by repeatedly predicting the next token. Temperature and top-p influence how those next-token choices are made.

These concepts provide a strong foundation for understanding how modern Large Language Models process language, learn from data, and generate responses.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. arXiv:1706.03762.
2. OpenAI. *What are tokens and how to count them?* OpenAI Help Center.
<https://help.openai.com/en/articles/4936856-what-are-tokens>
3. Hugging Face. *Fine-tuning*. Hugging Face Transformers Documentation.

<https://huggingface.co/docs/transformers/en/training>

4. Hugging Face. *Text Generation*. Hugging Face Transformers Documentation.

https://huggingface.co/docs/transformers/main/main_classes/text_generation